

Urdu Sentiment Corpus (v1.0): Linguistic Exploration and Visualization of Labeled Dataset for Urdu Sentiment Analysis

Muhammad Yaseen Khan
Center for Language Computing
Department of Computer Science
Mohammad Ali Jinnah University
Karachi, Pakistan
yaseen.khan@jinnah.edu

Muhammad Suffian Nizami
Center for Language Computing
Department of Computer Science
Mohammad Ali Jinnah University
Karachi, Pakistan
m.suffian@jinnah.edu

Abstract—The significance of the labeled dataset is not obscure from artificial intelligence practitioners. We have seen much phenomenal work, in natural language processing, for many languages (like English, Chinese, and Arabic, etc.), due to the reason for the availability of substantial data. For the Urdu language, despite the third largest spoken language in the world, very little research work is shown; hence, it is adjudged as a ‘morphologically rich’ but ‘resource-poor’ language. Further, the researchers working on Urdu natural language processing are in a quandary due to the lack of availability of labeled/annotated datasets. This paper shares the data, “Urdu Sentiment Corpus” (USC), and insights therein, of Urdu tweets for the sentiment analysis and polarity detection. The dataset is consisting of tweets, such that it casts a political shadow and presents a competitive environment between two separate political parties versus the government of Pakistan. Overall, the dataset is comprising over 17,185 tokens with 52% records as positive, and 48% records as negative. This paper shares the visual insights (from document-level to word-level) into the textual similarities, manifold-learning, etc. In addition to it, this paper also presents a Part-of-Speech wise analysis and an unpretentious technique for the extraction of sentiment lexicons from the corpus.

Index Terms—Corpus, Dataset, Computational Linguistics, Linguistics, Sentiment Analysis, Sentiment Classification, Urdu, Visualization.

I. INTRODUCTION

Amongst 7K+ different languages spoken in the world, the Urdu language with its 329.1M native speakers and 697.4M total speakers all around the world stands at the 3rd place [1]. The origin and birth of the Urdu language can produce a fascinating yet extensive discussion. On a short note, Urdu is evolved during the medieval era of 6–13th century in India. It is accounted as Middle Indo-Aryan language [2], and claimed to be a creole language through the fusion of various Asian and European languages¹ [3]. Around 75% of Urdu words have their etymological roots in Sanskrit and Prakrit [4], [5] and

approximately 25% words came into the Urdu vocabulary as the result of Arabic and Persian contributions [4], [5], [6], [7], [8]. “Hindi” is a mutually intelligible twin/alternate language of Urdu [9], [10], and it is shown that the morphological system of both languages is almost same [11], [12]. Regardless of the strong morphological and lingual bondage, these languages are the victim of language split, whose chronicle revolves around the era of the independence of the Indian sub-continent from the British Raj in its final decades [13], [14]. In the world of today, the major differences between these two languages are of writing script and the dominance of specific vocabulary. Urdu is written in Perso-Arabic script and shares a large portion from Arabic and Persian vocabulary [15], [16]. Whereas, Hindi is considered to be written in the Devanagari script and exclusively uses the Sanskrit vocabulary [15]. The word “Hindustani” is considered as the common word to represent both of these languages [17], and Urdu and Hindi are considered as the Persianised and Sanskritised registers of Hindustani language [18], [19].

Coming in expressing sentiments in one’s mother-tongue is an easy and convenient task [20]. The Urdu enthusiasts went a long mile before enjoying the reading and writing the text in Urdu Unicode on computers. Apparently, with the beginning of computing (and the internet), there were very few opportunists who know to write in Urdu on computers. Later, with the advancement in Urdu text-processing software (started in the 1970s [21]), people started to compose text in Urdu languages. Currently, many operating systems and web-solutions offer built-in language settings for the Urdu language, which makes writing in Urdu more handier for the people.

The human nature is a curious metaphor [22]. It wills to perfect things whatever comes to its satisfaction, so with choice of words for the expressions of its sentiment and emotions [23]. Many of us think: Is there exist any difference between sentiments and emotions? A simple answer is yes [24]. “Sentiment” (S) relating to a running text can be of two

¹like Arabic, Persian/Farsi, Sanskrit, Bengali, Sindi, Punjabi, Pushto, Turkish, Portuguese, French, and English.

types i.e. either *Positive* (**P**) or *Negative* (**N**), $S \in \{P, N\}$ [25]. Whereas, the ‘Emotions’ are far beyond being binary in status. *Anger, Disgust, Fear, Happiness, Sadness, and Surprise* etc. are to name a few of basic emotions [26].

The invention of the internet has made a profound impact on the human lifestyle. Moreover, with the exponential growth of internet usage, a lot of mobile and web applications have replaced the physical routines of work [27], [28]. The online social networks (OSN) such as Facebook and Twitter are unarguably the most successful mediums of opinion sharing [29], where the people remotely share opinions and express their emotions on the ongoing trends in their domains of interests. In the past two decades, researchers in the communities of natural language processing (NLP) and psychology are exploring the text on the internet [30], [31]. One of these exploratory tasks is to analyse the affect (sentiment analysis and emotion detection) in the running texts of OSN posts [32], [33], [34], [35]. Constructively, these sort of tasks is powerful tools in strategic decisions and policymaking.

This paper presents a dataset, which is the collection of 1,000 tweets² for sentiment analysis and polarity detection in Urdu running text. These tweets nearly present a monolingual corpus written in standard Urdu, i.e., Perso-Arabic script. Computational models for committing such tasks are considered as a classification problem and a wide range of approaches and techniques have been employed for sentiment analysis such as Lexicon or Rule-based (e.g. [36], [37]), Supervised Learning (e.g. [38], [39], [25], [40] etc.), Unsupervised Learning (e.g. [41], [42] etc.), Semi-Supervised Learning (e.g. [43], [44] etc.), and Hybrid Techniques (e.g. [45], [46] etc.).

The presented paper is further divided into nine sections. Section II gives the historical background, the methodology of data collection, cleaning, and data-labeling. Section I presents the detailed analysis, visualization of the data w.r.t the class distribution of, length of documents and words, and Mean Absolute Error among the classes. Section IV shares the evaluation of the dataset under the empirical Zipf’s law. Section V shows the evaluation of dataset on the ground of textual similarities, it also contributes to the averaging of Sørensen–Dice and Tanimoto coefficients within in Tversky index. The same section deals with the generalized algorithm for Term-Document-Incidence construction for n -grams. Section VI presents the visualization of the entire dataset with manifold learning. Section VII gives exciting insight into the association of words with respective classes and also discusses the criterion for building a sentiment lexicon with Pointwise Mutual Information. Section VIII presents the Part of Speech (PoS) analysis with universal PoS and language-specific PoS tags. In last we have a conclusion, and future work followed references in the end.

II. THE DATASET

This section shares the chronicle details of the dataset, the methodology of data collection, challenges, and procedure of dataset cleaning, followed by the mechanism of data labeling.

A. Historic Background

The tweets are collected between 14 August–17 December 2014. Over the mentioned period, the political parties in Pakistan, in particular, Pakistan Tehreek-e-Insaf (PTI) and Pakistan Awami Tehreek (PAT) were protesting in Islamabad against the government. PTI, in the leadership of Imran Khan Niazi, accused and protested against the rigging in the 5th general election of Pakistan (held in 2014) that later turned into the protest against the offshore assets (as mentioned by Panama Leaks) and corruption of former Prime Minister (PM) Mian Muhammad Nawaz Sharif. PAT, in the leadership of Tahir ul Qadri, was protesting against the government’s unjustified action taken on 11th May 2014 at Model Town in Lahore, Pakistan. The leadership of both of these protests announced the end of these protests on the most horrific and saddest incident of the terror attack on Army Public School, Peshawar. Although the data presented in the dataset exhibit a competitive political attire, we cannot say the political showcase is dominating in the entire dataset.

B. Data Collection and Challenges

The data, i.e., tweets are collected by using an API; namely, *tweet4j*³. To an approximate 7K tweets were collected in that are in Urdu. Although it can be seen as in overwhelming size, there are major challenges in dealing with the data, which are listed below:

- 1) **ReTweets.** A significant portion of the collected tweets is a facsimile. Although Twitter marks these shared tweets as *retweet* in the database, we cannot do anything in the situation where a user copies a tweet and post it on Twitter, which is not marked as a retweet in database.
- 2) **Anchors.** These are the shortened URLs. In most of the cases, these are found as a spamming/marketing tactic, used in-reply-to the trending tweets.
- 3) **Informing Mentions.** This is the method by which a Twitter user can connect the other Twitter fellow on a tweet. For doing so, the user needs to tag the fellow’s Twitter handle (a user-name written with @ sign in the start, followed by alpha-numeric word) in reply to the original tweet. Putting only a mention in reply is the other form of data duplication.
- 4) **Hash-Tags.** This is the most troublesome challenge in the collected dataset. A hash-tag is defined as a word or a phrase (without whitespaces) starting with a # sign, written to characterize and associate any text/tweet with a trend/topic. Likewise mentions, hash-tags can produce a redundant set of tweets, if these are going to be added in replies.
- 5) **Word Segmentation.** One of the most challenging problems in Urdu (or any other language with Perso-Arabic script such as Arabic, Sindhi, or Punjabi (in Shahmukhi script)) is to deal the word segmentation. It usually happens when a user (intentionally or unintentionally) missed inserting whitespace after the non-joiner Urdu

² We use the words *tweet* and *document* interchangeably.

³ Details can be sought at <http://twitter4j.org/en/>.

alphabet. As a non-joiner alphabet is not going to change the form of the next alphabet, thus the reader can read the word/phrase without any hurdle, but for the computational task, the entire phrase is considered as a single token. For example: the phrases طاهر القادري /tāhir al.qādiri/ and محمد نواز شريف /mu.ḥam.mad navâz šarīf/ are consisting of multiple distinct tokens, طاهر /tāhir/ and القادري /al.qādiri/ in first phrase; محمد /mu.ḥam.mad/, نواز /navâz/, and شريف /šarīf/ in the later one. It happened due to د and ز in the second phrase and ر in both phrases are non-joiners and alphabets following them will not change themselves into their intermediate and final forms [36].

C. Data Cleaning and Dighting

Following the completion of tweets collection activity, the procedure of data cleaning begins. The issues, as we have seen in the previous subsection, are resolved in two steps. Thus, for a tweet (τ), the first step involves the removal of URL anchors, Twitter handles, and hash-tags therein, through the utilization of Regular Expressions (RegEx), such that doing so will transform τ into the modified tweet ($\Delta\tau$). Let $\Theta(\beta, \tau)$ be the function that takes a RegEx pattern β and text τ , and gives you the $\Delta\tau$ in return. In the next step, $\Delta\tau$ is inducted to a hash-set ($\mathbf{H}$) to gather the unique set of tweets. Algorithm 1 defines the whole process applied to the entire collection of tweets $\mathbf{T}$, such that $\mathbf{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$.

Algorithm 1: Data cleaning and redundancy removal.

Result: Set of distinct tweets $\mathbf{H}$.
 $\mathbf{T} \leftarrow$ be the set of tweets;
 $\mathbf{H} \leftarrow$ be an empty hash-set;
 $\mu \leftarrow$ be the RegEx pattern for URLs;
 $\alpha \leftarrow$ be the RegEx pattern for Twitter handles;
 $\lambda \leftarrow$ be the RegEx pattern for hash-tags;
for every τ **in** $\mathbf{T}$ **do**
 $\Delta\tau \leftarrow \Theta(\lambda, \Theta(\alpha, \Theta(\mu, \tau)))$;
 $\mathbf{H} \leftarrow \mathbf{H} \cup \{\Delta\tau\}$;
end

We have employed the hash-set because, under hash-function, the redundant tweets collide to have the final result in the form of the set of distinct tweets. Dictionaries or Hash-maps can be used as an alternative to hash-sets.

Lastly, we tokenized the dataset and extracted the distinct words in the corpus. Then, we manually performed the segmentation on the tokens, where space was not inserted after non-joiners. For doing so, we have utilized algorithm 2, which gives us a distinct set of words in a dictionary format. The algorithm 2 is discusses in details in §V.

D. Data labelling

Each tweet in the dataset was labeled by three judges, whether it is Positive (**P**), Negative (**N**), or Objective (**O**). The Objective is said to the thing which does not possess any sentiment. The final label was considered positive where the

tweet got two positive votes, so with the negative, otherwise, it was called Objective. Mathematically,

$$f(\mathbf{L}_1^\tau, \mathbf{L}_2^\tau, \mathbf{L}_3^\tau) = \begin{cases} \mathbf{P} & \sum_{i=1}^3 \mathbf{L}_i^\tau = \mathbf{P} \geq 2\mathbf{P} \\ \mathbf{N} & \sum_{i=1}^3 \mathbf{L}_i^\tau = \mathbf{N} \geq 2\mathbf{N} \\ \mathbf{O} & \text{Otherwise} \end{cases} \quad (1)$$

where $\mathbf{L}_1^\tau, \mathbf{L}_2^\tau$ and $\mathbf{L}_3^\tau$ are individual labels given by judges $\mathbf{L}_1, \mathbf{L}_2$ and $\mathbf{L}_3$ to tweet τ . The authors and judges were not aligned/agree/disagree with the tweets and persons whom they belong to. Secondly, form this activity, we also generated a matrix (Ψ) as follows, which shows the percent of agreement among judges for a given tweet τ ; wherein Ψ , the row contributes the documents/tweets, and the columns are the labels.

$$\Psi = \begin{matrix} & \begin{matrix} \mathbf{P} & \mathbf{N} & \mathbf{O} \end{matrix} \\ \begin{matrix} \tau_0 \\ \tau_1 \\ \tau_2 \\ \vdots \\ \tau_N \end{matrix} & \begin{pmatrix} .66 & 0 & .33 \\ .33 & .66 & 0 \\ .33 & .33 & .33 \\ \vdots & \vdots & \vdots \\ .66 & .33 & 0 \end{pmatrix} \end{matrix}$$

Moreover, to compute the inter-agreement between the judges, we have used the Fleiss' kappa statistic (κ) [47]. The Fleiss' kappa statistic is the generalized form of Scott's pi statistic [48], and relatively enhances the Cohen kappa statistic [49], to measure the inter-agreement reliability for any number of judges. The Fleiss' kappa statistic for our labeling activity is 0.63, which shows the substantial agreement among judges.

In order to compute the Fliess' kappa, consider the matrix Ψ above, and further we can see the following equations; for which, consider N be the total number of documents, k be the number of categories that in our case is either of $\{\mathbf{P}, \mathbf{N}, \mathbf{O}\}$; i and j are the referring indexes for N and k respectively where $i = \{1, \dots, N\}$ we first calculate p_j and $j = \{1, \dots, k\}$. We first calculate p_j . i.e., the proportion of all assignments which were to the j^{th} category:

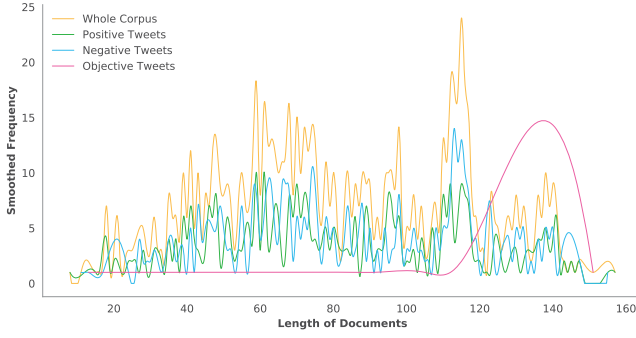
$$p_j = \frac{1}{Nn} \sum_{i=1}^N n_{ij}, \quad \text{provided that } 1 \approx \sum_{j=1}^k p_j \quad (2)$$

Now calculate P_i , the extent to which judges agree for the i^{th} class:

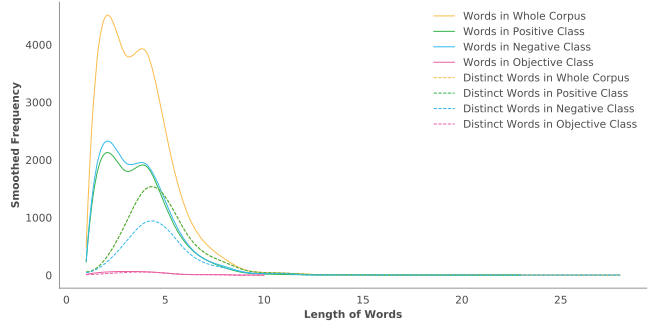
$$\begin{aligned} P_i &= \frac{1}{n(n-1)} \sum_{j=1}^k n_{ij}(n_{ij} - 1) \\ &= \frac{1}{n(n-1)} \sum_{j=1}^k (n_{ij}^2 - n_{ij}) \\ &= \frac{1}{n(n-1)} \left[\left(\sum_{j=1}^k n_{ij}^2 \right) - n \right] \end{aligned} \quad (3)$$

now compute $\bar{P}$ i.e., the mean of the P_i 's; and the $\bar{P}_e$

$$\bar{P} = \frac{1}{N} \sum_{i=1}^N P_i = \frac{1}{Nn(n-1)} \left(\sum_{i=1}^N \sum_{j=1}^k n_{ij}^2 - Nn \right) \quad (4)$$



(a) Document-length frequency plot.



(b) Word-length frequency plot.

Figure 1: Plots of document-length and word-length and their frequencies. In figure 1b, frequency for unique/distinct words is also calculated.

$$\bar{P}_e = \sum_{j=1}^k p_j^2 \quad (5)$$

and finally through the equation 4 and 5 we can calculate κ as follows:

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e} \quad (6)$$

III. CLASS DISTRIBUTION AND MEAN STATISTICS

Table I: Class distribution in dataset and word frequencies.

Class	Document Count	Word Count	Distinct Word Count
Positive	480 (48%)	8,184 (47.62%)	3,075 (58.60%)
Negative	500 (50%)	8,748 (50.90%)	3,269 (62.30%)
Objective	20 (2%)	253 (1.47%)	182 (3.65%)
Total	1,000	17,185	5,247

In total, there are 1000 tweets, comprising over 17,919 tokens. Table I gives the details in numbers and percentage share w.r.t the class for document and token count accordingly. However, the column distinct word count in table I shows the contribution of words in respective classes regardless of their appearance in other classes.

Since the authors have mentioned at the beginning of this section that for some analyses the objective tweets are considered as positive, thus, in doing so the number of positive documents will become 500, grossing the 50% of the whole dataset with 8,231 tokens. Hence, we can have an almost class balanced dataset. For the ease of reporting convention, we would like to use the symbol $\mathfrak{R}$ for referring the (real) dataset with positive, negative, and objective tweets accounted as separate classes; while symbol $\mathfrak{I}$ for referring the (imagined) dataset where objective tweets are treated as negative. The average character length of the documents in a dataset is $81.22 \approx 81$. Similarly, the average character length of tokens in a dataset is $4.01 \approx 4$. Figure 1a shows the frequency and the length of documents, for the whole corpus and as well as for individual classes; and figure 1b shows the length of words and distinct words in the same fashion. Moreover, the Mean Absolute Error (MAE) which is the statistic for showing the

average absolute difference between two adjacent points in a continuous series [50], such that,

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - x_i| = \frac{1}{N} \sum_{i=1}^N |e_i| \quad (7)$$

where y_i and x_i are the length of i^{th} positive and negative tweet, respectively, and N is the total number of tweets. e_i briefly corresponds to the error i.e. $y_i - x_i$ for i^{th} tweet. Figure 2 shows absolute difference along the sequence of tweets, and MAE w.r.t $\mathfrak{R}$ and $\mathfrak{I}$. The MAE of positive and negative tweets in a balanced distribution (yielded from the plot of figure 1a) is found 5.58 and 5.77 respectively, which shows the fact that at average, a document may have the difference of $\pm[1, 2]$ words with any other document in dataset.

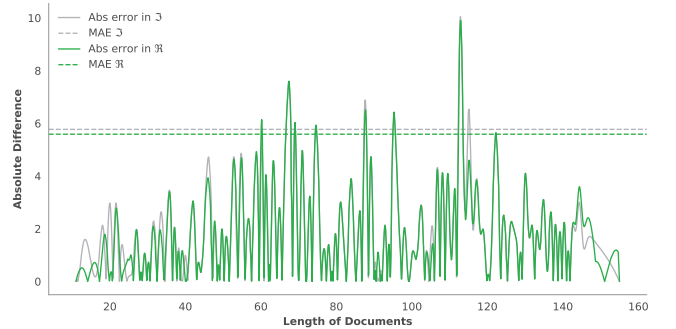


Figure 2: MAE plot of positive and negative tweets in $\mathfrak{R}$ and $\mathfrak{I}$.

IV. EVALUATION OF CORPUS UNDER EMPIRICAL LAWS

185 *Zipf's law*—as it is one of the most famous empirical laws amongst the quantitative linguists, observed and formulated in the 1930s and named after the linguist George Kingsley Zipf [51], [52]; however, Powers maintained a statement that Zipf did not claim the law [53]. In the same study [53] and similarly others (for example, [54], [55]), the authors have maintained the applications of Zipf's law in various domains

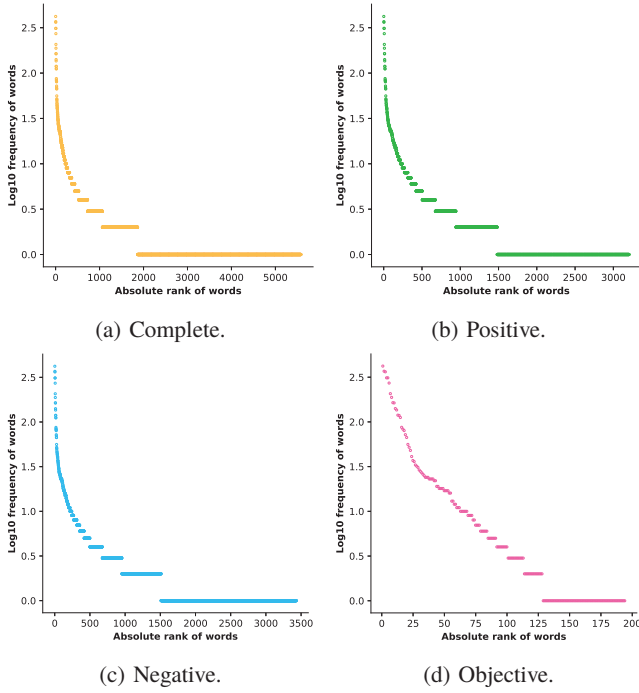


Figure 3: Zipf's plot for the whole corpus, and documents corresponding individual classes respectively.

of science. However, in the field of linguistics, the law is contributed to the relationship of word appearance and its rank in the corpus; in simple words, it states the frequency-rank of a word in the corpus is inversely proportional to its frequency. Specifically,

$$g(k; s, N) = \frac{1/k^s}{\sum_{n=1}^N (1/n^s)} \quad (8)$$

where N be the number of elements; k be their rank; s be the value of the exponent characterizing the distribution.

An alternative intuition of the Zipf's law says that the i^{th} most common term in the corpus tends to have the document-frequency df_i as inversely promotional to i [56], mathematically,

$$df_i \propto \frac{1}{i} \quad (9)$$

such that the most frequent word in the corpus (i.e., with frequency-rank 1) is likely to appear in every document, and the second most frequent word (i.e., with frequency-rank 2) in half of the documents, and so on. This trait produces a desideratum to run an experiment, which turned out in a remarkable observation that the top 13339 distinct words w.r.t to their frequency-ranks subsume almost half of the tokens in the entire corpus. Historically, this experiment was first conducted on the Brown corpus [57]⁴ for the English language, and observation concluded that merely top 135

distinct words are required for nearing the half of Brown corpus tokens/words [58], [59].

Figure 3 shows the log-log Zipf's plot for the complete corpus, along with the visualization of law for the individual classes. Furthermore, for every subfigure in figure 3, at the x-axis of the plots shows the absolute rank of the words, and at the y-axis, we have the $\log_{10}$ of the frequencies. It looks similar to visualize curves in plots (see figures 3a, 3b, and 3c), but in reality, these figures portray the frequencies ranks as per the actual number of distinct words w.r.t the classes i.e. 5, 327, 3, 093, and 3, 303 in whole corpus, positive tweets, and negative tweets respectively. These counts are almost similar, especially the last two, hence the plots appear similar. Figure 3d shows a difference because the number of distinct words are too few (i.e., 173) in comparison to the rest of all.

V. TEXTUAL SIMILARITIES

It would come rational to ponder that whether there exists any similarity amongst the tweets in the corpus or not? Although, we have seen the removal of duplicate/redundant tweets from the collection in the algorithm 1 (in §II-C), still, are there any textual similarities based on tokens? In this regard, to answer the query, textual similarities can be measured with a lot of techniques, for example Jaccard index [60], Sørensen–Dice coefficient [61], [62], Tversky index [63], Tanimoto distance [64], Cosine similarity, and Monge–Elkan [65] are used for calculating token-based similarity and/or distances. A quite trade-off between *distance* and *similarity* indices is of the inverse proportionality: as the distance between two things increases, it will cause decrement to the similarity between them, and vice-versa. In this work, we consider the Tversky index to measure similarity in tweets. A big motivation behind choosing the Tversky index can be thought of it being as a generalization of Sørensen–Dice's coefficient, and Tanimoto coefficient (which in some cases, also regarded as Jaccard index). The Tversky index needs two sets (say X and Y) to calculate asymmetric similarity measure, such as mathematically:

$$S(X, Y) = \frac{|X \cup Y|}{|X \cap Y| + \alpha|X - Y| + \beta|Y - X|} \quad (10)$$

where $X - Y$ is the relative complement of Y in X ; and $\alpha, \beta \geq 0$ are the parameters in Tversky index. Since, it is the generalization of Sørensen–Dice's coefficient, and Tanimoto coefficient, therefore, if we set the values of these parameters as $\alpha=\beta=1$ then it would become Tanimoto coefficient, whereas, setting the values to $\alpha=\beta=0.5$ will produce Sørensen–Dice's coefficient. However, the baseline functions of Tanimoto similarity (T_s) and Sørensen–Dice's coefficients (SD_c) are given in equation 11 and 12 respectively.

$$T_s(X, Y) = \frac{\sum_i (X_i \wedge Y_i)}{\sum_i (X_i \vee Y_i)} \quad (11)$$

$$SD_c(X, Y) = \frac{2|X \cap Y|}{|X| + |Y|} \quad (12)$$

⁴<http://clu.uni.no/icame/manuals/brown/index.htm>

		Sørensen–Dice (SD_c)			Tanimoto Sim. (T_s)			$M = \mu(SD_c, T_s)$			Tversky Index (T_i) $\alpha = \beta = 0.7$			$\Delta = \ M - T_i\ $			$\mu\Delta$
		P	N	O	P	N	O	P	N	O	P	N	O	P	N	O	
Chars.	P	.5375			.3822			.4598			.4614			.0016			.0039
	N	.5485	.5390		.3841	.3946		.4663	.4668		.4633	.4736		.0030	.0068		
	O	.4470	.4514	.4928	.3044	.3055	.3600	.3757	.3784	.4264	.3762	.3748	.4186	.0005	.0036	.0079	
	W		.5389			.3843			.4616			.4634			.0017		
uni-grams	P	.0426			.0244			.0335			.0324			.0011			.0020
	N	.0388	.0452		.0204	.0258		.0296	.0355		.0285	.0343		.0011	.0012		
	O	.0222	.0191	.1177	.0118	.0100	.1072	.0170	.0146	.1124	.0162	.0148	.1046	.0008	.0002	.0079	
	W		.0412			.0227			.0319			.0308			.0011		
bi-grams	P	.2107			.1223			.1665			.1631			.0035			.0044
	N	.2094	.2164		.1200	.1261		.1647	.1712		.1613	.1678		.0034	.0034		
	O	.1614	.1559	.2369	.0905	.0867	.1737	.1260	.1213	.2053	.1224	.1182	.1956	.0036	.0031	.0096	
	W		.2102			.1212			.1657			.1623			.0034		
tri-grams	P	.0600			.0334			.0467			.0451			.0016			.0153
	N	.0565	.2164		.0296	.0346		.0431	.1255		.0415	.0468		.0016	.0787		
	O	.0374	.0366	.1314	.0194	.0191	.1142	.0284	.0278	.1228	.0273	.0272	.1143	.0012	.0006	.0085	
	W		.0584			.0316			.0450			.0434			.0016		

Table II: Text similarity measures w.r.t the comparison on character-level, and n -grams up to $n=3$. The rows/columns corresponding to the alphabets P, N, and O show the classes; hence, the cell with row-title P and column-title P shows the inter-document similarity of documents belong to P class and so on; whereas, W shows the similarity measure for the whole corpus.

Algorithm 2: Building universal set of distinct tuples.

Result: A dictionary (U) with the tuples as keys and respective index number as values.

```

T ← be the set of distinct tweets;
U ← be the empty dictionary;
c ← be the counter variable set to 0;
n ← be the window of words w.r.t  $n$ -grams;
for each tweet  $\tau$  in T do
    T ← split  $\tau$  into tokens;
     $\ell \leftarrow |T| - n$ ;
    for each  $k \in \{0, \dots, \ell\}$  do
         $\delta \leftarrow \langle T[k : k + n] \rangle$ ;
        if  $\delta \notin U$  then
            U[ $\delta$ ] ← c;
            c ← c + 1;
        end
    end
end
end

```

Before we proceed to explore the results of text similarity measures, we would like to account two different approaches (hypotheses) to test and measure the corpus, i.e., (a) instead of running Sørensen–Dice and Tanimoto coefficients separately and further taking an average of their results, we adjusted the parameters in Tversky index as $\alpha=\beta=0.7$ in order to examine the behavior of Tversky index such that can it show-up an average of Sørensen–Dice and Tanimoto coefficients?; and (b) we have constructed the Term-Document Incidence (TDI) up to tri-grams (considering the n -gram feature where $n=3$) to verify the first hypothesis thoroughly.

The TDI is a matrix representation of documents and their respective n -gram in the form of tuples, more specifically, it is a non-sparse matrix showing (at least) boolean representation of documents in 2-dimensional vector space, such as the rows

Algorithm 3: Construction of term-doc incidence.

Result: A non-sparse matrix showing boolean representation of documents in n -dimensional vector space.

```

U ← be the dictionary resulting from algorithm 2.;
I ← be the empty list (to nest the vectors of documents);
u ← be the size/length of T;
c ← be the counter variable set to 0;
n ← be the window of words w.r.t  $n$ -grams;
for each tweet  $\tau$  in T do
    X ← be the local list of  $u \times false$ ;
    T ← split  $\tau$  into tokens;
     $\ell \leftarrow |T| - n$ ;
    for each  $k \in \{0, \dots, \ell\}$  do
         $\delta \leftarrow \langle T[k : k + n] \rangle$ ;
        X[U[ $\delta$ ]] ← true;
    end
    I.append(X)
end

```

of the matrix show the documents (in the form of vectors) and the columns represent the tuples of tokens, the boolean values corresponding to a row-column indicates existence of particular tuple in document or not (see the matrix TDI below)

$$TDI = \begin{matrix} & t_0^n & t_1^n & \dots & t_{|U|-1}^n \\ \begin{matrix} \tau_0 \\ \tau_1 \\ \vdots \\ \tau_N \end{matrix} & \begin{pmatrix} false & false & \dots & false \\ false & true & \dots & true \\ \vdots & \vdots & \ddots & \vdots \\ true & false & \dots & false \end{pmatrix} \end{matrix}$$

where t_i^n is the tuple of tokens, such that i represents the index of tuple in the Universal set of distinct tokens (U), and n is the size of tuple w.r.t the n -gram; provided that $n \geq 1$ and $i = \{0, \dots, |U| - 1\}$. Further, its mathematical description,

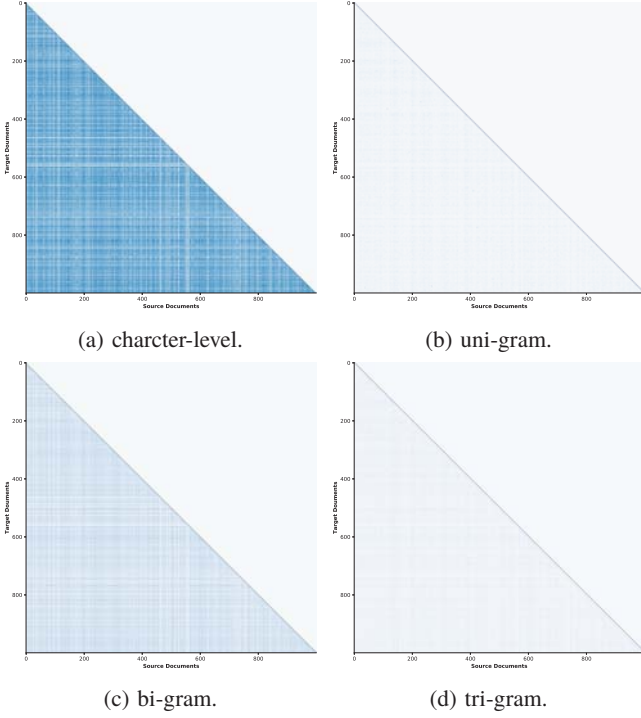


Figure 4: Plot of inter-document similarity on whole corpus with respect to parameter values of Tversky index as $\alpha = \beta = 0.7$.

w.r.t n -gram for the document having l number of words, can be $t_i^n = \langle w_{j=k}, \dots, w_{j=k+n-1} \rangle$, where j is the index of word in document, and k be the starting and ending value of j , provided $0 < j < l$ and $0 < k \leq (l - n)$. For the construction of TDI, we propose a slightly modified approach (in contrast to the algorithm presented in [66]) to accommodate the n -gram features, instead of producing a plain TDI which shows documents and terms i.e., uni-grams. The algorithm 2 defines the construction of universal set of distinct tuples ($\mathbf{U}$). Once, we have constructed $\mathbf{U}$, we would employ it in construction of TDI as mentioned in algorithm 3, hence the algorithm 2 will be the pre-requisite for the TDI construction.

Coming to the discussion of text similarity, we can see the results in table II, where every tweet is measured with the rest of the tweets w.r.t the classes and the whole dataset. Further, the similarity of document X and Y are symmetric ($\Phi(X, Y) = \Phi(Y, X)$, Φ is either of the similarity functions); thus, the similarity between two classes is reported once. Hence, we have gained a thorough insight into the inter-document similarity with three different algorithms. The column M shows the averaged SD_c and T_s values, which is further used to take the absolute difference with T_i (see column with title $\Delta = \|M - T_i\|$), followed by mean-difference ($\mu\Delta$) in the next column. Moreover, the hypotheses (as stated in §2-§V that, since the T_i is the generalization of SD_c and T_s ; so that, if we adjust α and β parameters of Tversky index as $\alpha = \beta = 0.7$ then, a near-average results of SD_c and T_s is expected) are

seen valid through the quantified results such that: $\mu\Delta$ at the character-level similarity is $\approx 0.4\%$, and for the n -gram we can see the inter-class mean-difference ($\mu\Delta$) is $\approx 0.2\%$, ≈ 0.4 and $\approx 1.5\%$ for uni-gram, bi-grams, and tri-grams respectively. At this point, we can surmise that as the n -gram increases, the $\mu\Delta$ between $\mu(SD_c, T_s)$ and T_i increases; and we need to readjust α and β parameters for different n -grams. Whereas, irrespective of the inter-class similarities, the inter-document similarity for the whole corpus shows a less difference in comparison to the inter-class mean-difference. We can see the difference of averaged Sørensen–Dice–Tanimoto measures and Tversky index is minute enough to gross around 0.11% , 0.34% , and 0.16% for uni, bi, and tri-grams respectively. In the same context of textual similarities, one can expect two things: first, documents belong to a particular class should show more similarity in contrast to the inter-class document similarity. Secondly, the overall similarity in the whole corpus should be minimum. We can see, in general, in the table II that the values (for a particular metric/similarity function under consideration; presented in the lower-triangle) at the diagonal are higher than the rest of values at the respective row.

Lastly, figure 4 shows the visualization for inter-document similarity on the whole corpus, where the darkness of the cell color is directly proportional to the similarity of corresponding documents, thus the diagonal shows the similarity of a document with itself, where the similarity is expected to be maximum. Since, the table II presents the inter-document similarity (w.r.t T_i) for the whole corpus as character-level is 46.34% , uni-gram is 3.08% , bi-gram is 16.23% , and tri-gram is 4.34% similar, therefore, accordingly with these values, we can see the figure 4a renders similarities in more darker color in contrast to the rest of sub-figures respectively.

VI. MANIFOLD LEARNING

In §V, we have seen the matrix representation of the corpus, i.e., Term-document-incidence (TDI), where every document (a row) is expressed in the form of a boolean vector, and elements in the vector are designated for the terms as per the universal set of distinct terms ($\mathbf{U}$). Further, in a generalized Euclidean space⁵, we consider every element of the vector as it embodies a separate dimension. Thus, the length of a vector in TDI is equal to the number of distinct words in the corpus, or similarly, a vector has dimensions as per the number of distinct terms in the corpus. It is very abstract to visualize, and difficult to interpret these vectors which are prodigious, and expressed in high-dimensional space. Consequently, the ‘manifold learning’ and ‘nonlinear dimensionality reduction’ algorithms are devised to resolve this problem. These techniques transform and map the vector data from/to a higher-dimensional space to/from low-dimensional space;

⁵A baseline Euclidean space, as it was introduced by Euclid[67], is consisting of three dimensions (representing x , y , and z axes) to explain space of physical objects in geometry; however, mathematical developments in 19th century brought to us more sophisticated perspective of geometric space, one of which is described in the seminal work of Schläfli [68], for its generalization to the high-dimensional space.

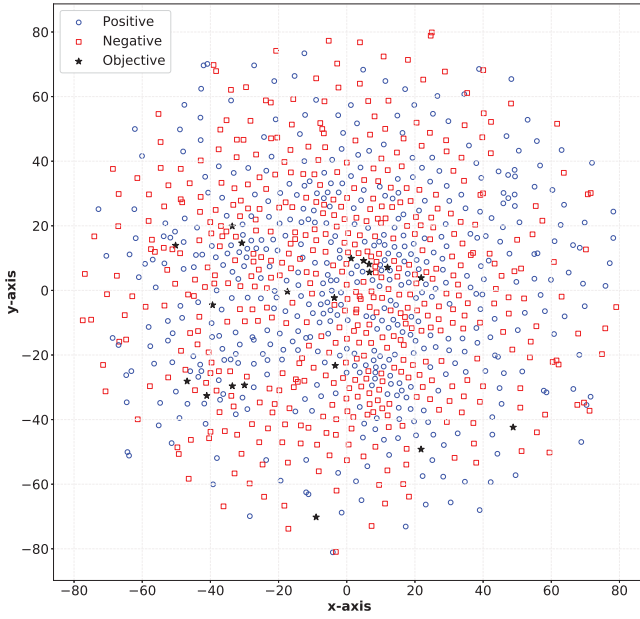


Figure 5: t-SNE visualization of the corpus.

thus, the transformed data in the 2/3-dimensional space can be visualized easily.

Through the mathematical applications in computer science, we have got a variety of such manifolding and dimensionality reduction algorithms, for example Locally-linear embedding [69], Laplacian eigenmaps [70], Sammon's mapping [71], Principal Component Analysis (PCA) [72], [73], Autoencoders[74], [75], t-distributed Stochastic Neighbor Embedding (t-SNE) [76], and similarly others are to name a few. In this paper, we have selected t-SNE for visualizing the corpus data.

Since its debut, t-SNE⁶ showed up as an influential algorithm for the nonlinear dimensionality reduction. This technique is the best fit for the visualization of data through the transformation of high-dimensional data in a low-dimensional space of 2/3-dimensions. t-SNE works in two phases, it first computes the probability distribution (P) for the data-points (in our case vectors) in a high-dimensional space such that neighborhood of similar objects is retained, for example, the following equation will compute the probability $p_{i|j}$ between two objects x_i and x_j , with provision that for probability where $i = j$ is set as $p_{ii} = 0$

$$p_{i|j} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)} \quad (13)$$

then in the second step, t-SNE learns to map/embed the datapoints in low-dimensional space in a way that the probability distribution (Q) in lower-dimension reflects, as much as

possible, to the P. It measures the similarities $q_{i|j}$ between two points in the map y_i and y_j , with provision that $q_{ii} = 0$:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}} \quad (14)$$

where y_i and y_j are the low-dimensional and high-dimensional maps, respectively. Further, it utilizes heavy-tailed Student's t-distribution [77] with 1-degree freedom to overcome the crowding problem and for the measurement of the similarities between points such that dissimilar objects are modeled far apart in the map; and the Kullback-Leibler divergence (D_{KL}) [78] to minimize the difference between two distributions (i.e., P and Q) w.r.t locations of datapoints in low-dimension map as:

$$D_{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (15)$$

In this paper, we employed the Scikit-Learn's package⁷ for the t-SNE visualizations, which is shown in figure 5. We can see the documents having a positive class are shown in the blue circle; similarly, the documents having the negative class are in red squares, whereas the black stars are the neutral documents. The visualization also appears as it is very challenging for clustering tasks.

VII. CLASS ASSOCIATION

This section covers the term-level insights in the corpus. Firstly, we show word frequency within the whole corpus and subsets of documents w.r.t classes; later, we show a simple technique for building a sentiment lexicon to infer the word-class association through the Pointwise Mutual Information (PMI) [79].

A. Terms/Words and Frequencies

Calculating the frequencies of terms/words (TF)^{8,9} in the corpus appears as a simple task but, in actual it can draw various important aspects to deal with, for example in determining of relevancy of retrieved documents against a query in search engines[80], [81], and in induction of discriminative weight for terms within subsets of documents w.r.t classes[82], [83]. Furthermore, we can yield a quite better list of stopwords from the corpus under experiment, instead of utilizing a generic stop word list of the language.

For the calculation of TF, the main challenge is, as we have the differences in special characters; therefore, we cannot employ simple tokenization schemes of the English language. We tokenize the sentence with whitespaces and a set of special characters of both Latin and Perso-Arabic scripts (for example {brackets of all sort, question mark, exclamation marks, single

⁷<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

⁸In contrast to the §III which shows the frequency related to the document and word lengths, this section deals with occurrences of the word in the corpus and subsets w.r.t classes.

⁹It should be in notice that TF is also regarded as term-frequency w.r.t another critical metric in NLP, i.e., tf-idf (term-frequency×inverse document frequency) statistic. In this paper, we do not deal the TF as it is meant by tf-idf; instead we count the occurrences of the term in the entire corpus, and w.r.t classes where mentioned.

⁶<https://lvdmaaten.github.io/tsne/>

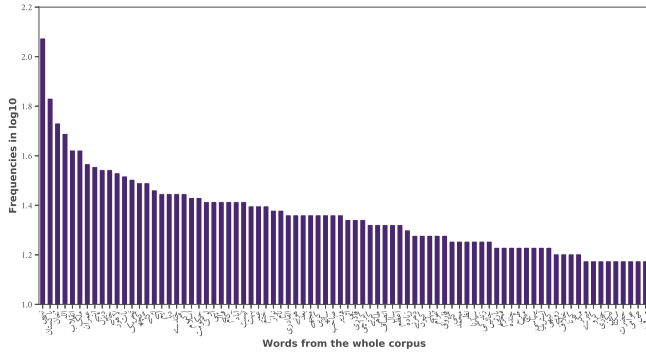


Figure 6: Top frequent word and their $\log_{10}$ frequencies.

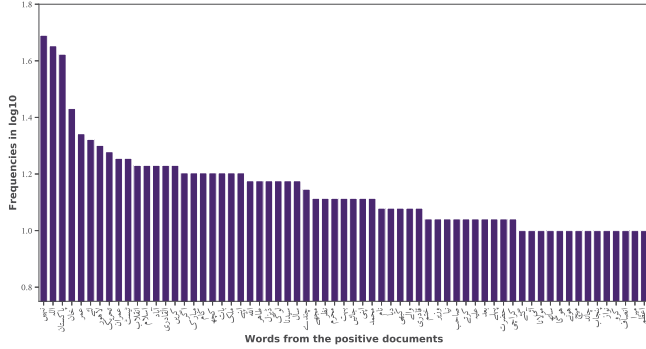


Figure 7: Top frequent word from tweets having positive class and their $\log_{10}$ frequencies.

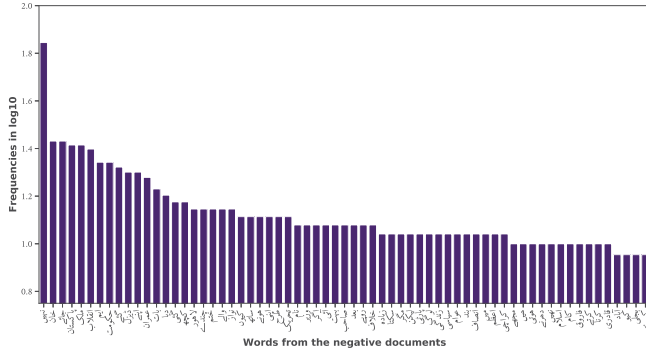


Figure 8: Top frequent word from tweets having negative class and their $\log_{10}$ frequencies.

and double quotes, mathematical signs, a hyphen, period, comma, colon, and semi-colon, etc.}. It also includes special characters (Unicode) for Arabic phrases that are widely used in Islamic text such as ﷻ, ﷺ, and ﷻ etc. In the next step, occurrences of the word are saved in a dictionary (where the word is the key). These two steps are performed thrice, i.e., for the whole corpus and the subsets of positive and negative documents, respectively. We also prepared the stop word list consisting of top 30 most frequent words; while eliciting these words, we dropped the proper nouns and negations in Urdu like نہیں /nə.fī:/ and نہ /nə/ etc (even if they are amongst top-frequent words).

Figure 6 shows the graph barplot of words in the whole

Table III: Differences in characters in Latin and Perso-Arabic scripts.

Character	Latin Script	Perso-Arabic Script
Question Mark	?	؟
Comma	,	،
Semi Colon	;	؛
Period	.	-
Decimal Point	.	ء

corpus. The TF are presented in $\log_{10}$ scale. Since there are thousands of words, which is technically very hard to visualize, therefore we have shown the tokens having TF at least 15 in the entire corpus. Figures 7 and 8 respectively shows the top 60 most frequent words in $\log_{10}$ scale within the $\mathcal{I}$ distribution of classes. Table III presents a brief survey of differences of special characters in Latin and Perso-Arabic scripts.

B. Construction of Sentiment Lexicon and Sentiment of Individual Terms

Although the term-frequencies w.r.t the classes can reveal important information, we cannot decide, mere on the frequency of the terms, whether a term is positive or negative. A big motivation behind it argues on a same term may appear at different positions in different subsets of the same corpus, for example the term عمران /imrān/ (Imran, name of person) appears at the 9th position and 12th position in subsets of positive ($\mathcal{I}$) and negative documents, so with the terms خان /xān/ (Khan, surname) which is at 4th and 2nd positions and انصاف /inṣāf/ (justice) which is at 65th and 48th position in positive and negative classes respectively. To see the words as mentioned earlier, kindly refer to the figures 7 and 8. Thus, we cannot specify the affinity and association of terms with the classes with the plain term-frequencies.

The Pointwise Mutual Information (PMI), as show in [84], comes as one of the probabilistic statistics through which we can identify the relationship between two terms or between a term and class. We can measure the PMI through the following equation.

$$\text{PMI}(w; c) \equiv \log \frac{p(w|c)}{p(w) \cdot p(c)} \quad (16)$$

where w is the term and c is the class. Through the same equation 16, we can also infer the sentiment score of a term as:

$$\text{SentiScore}(w) = \text{PMI}(w, \mathbf{P}) - \text{PMI}(w, \mathbf{N}) \quad (17)$$

Furhter, for the term-level sentiment analysis, we calculated the SentiScore for every word in the corpus (excluding stop-words), and the range of SentiScore is found as $[-1.0, 1.05]$; hence, we can assume a term is *positive* if $\text{SentiScore}(\cdot) > 0$ or *negative* otherwise. We found that out of 4,774 ditinct tokens (excluding stopwords), 2,467 (51.67%) tokens have positive SentiScore, whereas 2,307 (48.32%) tokens have negative SentiScore.

Figure 9 shows the visualization of term-level sentiments through a scatter-plot. The red-circles represents the words

Table IV: Words with their respective PMI values and score.

Word	Frequency	PMI		Sentiment Score
		Positive	Negative	
نہیں	119	-0.0843	0.0706	-0.1549
پاکستان	69	0.0854	-0.1065	0.1919
اللہ	49	0.2640	-0.7871	1.0512
ملک	45	-0.1481	0.1102	-0.2583
انقلاب	42	-0.0918	0.0757	-0.1675
لاہور	38	0.0830	-0.1027	0.1856
جائے	35	-0.3399	0.1883	-0.5283
ڈیزل	35	-0.0669	0.0580	-0.1249
بات	33	-0.0134	0.0130	-0.0263
تحریک	33	0.0613	-0.0714	0.1326
چھ	31	0.0138	-0.0142	0.0280
دبا	29	-0.0822	0.0691	-0.1513
حکومت	28	-0.3680	0.1963	-0.5643
ام	28	-0.3680	0.1963	-0.5643
لوگ	27	0.0458	-0.0512	0.0969
ٹیسٹ	27	0.1249	-0.1761	0.3010
آباد	27	0.1249	-0.1761	0.3010
اسلام	27	0.1001	-0.1303	0.2304
کراچی	26	-0.0726	0.0621	-0.1347
عمر	26	0.2285	-0.5119	0.7404
کام	26	0.0902	-0.1139	0.2041
والے	26	-0.0348	0.0322	-0.0669
ختم	25	-0.0555	0.0492	-0.1047
بہت	25	0.0170	-0.0177	0.0348
نواز	24	-0.0792	0.0669	-0.1461
مجھے	23	0.0532	-0.0607	0.1139
صاحب	23	-0.0193	0.0185	-0.0378
ساتھ	23	-0.0607	0.0532	-0.1139
القادری	23	0.1698	-0.2825	0.4523
ہونے	23	-0.0607	0.0532	-0.1139
وزیر	23	-0.0193	0.0185	-0.0378
انصاف	22	-0.0414	0.0378	-0.0792
طاہر	22	0.1627	-0.2632	0.4260
قادری	22	0.0378	-0.0414	0.0792
اعظم	22	-0.0414	0.0378	-0.0792
کرتے	21	0.0202	-0.0212	0.0414
سال	21	0.1549	-0.2430	0.3979
زیادہ	20	-0.0458	0.0414	-0.0872
کیوں	19	-0.1996	0.1362	-0.3358
عوام	19	-0.0746	0.0637	-0.1383
پہلے	19	0.0637	-0.0746	0.1383
دھرنے	19	-0.0235	0.0223	-0.0458
فاروق	19	-0.0235	0.0223	-0.0458
نظر	18	0.1597	-0.2553	0.4150
محمد	18	0.1597	-0.2553	0.4150
زندگی	18	-0.1091	0.0872	-0.1963
سیدنا	18	0.2218	-0.4771	0.6990
جائیں	18	0.1597	-0.2553	0.4150
پارٹی	17	-0.1513	0.1120	-0.2632
میچ	17	0.0706	-0.0843	0.1549
آسٹریلیا	17	0.0248	-0.0263	0.0512
محرم	17	0.1845	-0.3274	0.5119
چندہ	17	0.0248	-0.0263	0.0512
سیاسی	17	-0.1513	0.1120	-0.2632
کبھی	17	0.1498	-0.2304	0.3802
روپے	16	-0.3010	0.1761	-0.4771
حضریت	16	0.1761	-0.3010	0.4771
خلاف	16	-0.3010	0.1761	-0.4771

with negative SentiScores, whereas green-squares correspond to a positive one. In the same regard, the sizes of these markers show the TF. The x and y axes show $\text{PMI}(\cdot; \mathbf{P})$ and $\text{PMI}(\cdot; \mathbf{N})$ respectively. Since the words are overload with the same PMI scores at their respective axes, therefore, we did two things: added a little jittering to the of text/labels of words, and removed labels where the x and y values are ditto. The words appearing in the second and fourth quadrants are negative and positive, respectively. The color of the markers is set to the lower opacity such that the words overlapping at the same coordinates will get darker in visualization. Since the dataset is not very substantial, therefore, we can surmise the SentiScores of certain words, such as ایدھی /e:ɖʱi:/ (Edhi, name of a person/associated with famous philanthropist) and انصاف etc., will be better as the data increase.

In the similar way, table IV presents a sample of 50 words (sorted in descending order of TF), having $\text{TF} > 15$, provided that the terms with *zero* score, i.e. $\text{SentiScore}(\cdot) \equiv \text{PMI}(\cdot, \mathbf{P}) = \text{PMI}(\cdot, \mathbf{N}) = 0$, are not shown in the table. Entries in the table IV show a good reflection of words-class association such that, as a matter of fact, people tends to have a positive attitude towards religious text [85], hence, the terms like اللہ /a:l.ɬa:h/ (God), محمد /mu.ham.mad/ (Muhammad, name of prophet/person), and سیدنا /saj.jidi.nə/ ((literal) my sir/master) etc. have positive SentiScores 1.05, 0.41, and 0.69 respectively; whereas, words like انقلاب /in.qi.la:b/ (revolution), حکومت /hukʰu:ˈmətʰ/ (government), and دھرنے /ɖʱərne:/ (sit-in) have (sencibly) negative SentiScores as -0.16, -0.56, and -0.04 respectively.

VIII. PARTS-OF-SPEECH ANALYSIS

Parts-of-Speech (PoS) tags are the necessary components of any ‘language processing. A properly PoS tagged corpus is very helpful in different NLP tasks, for example understanding grammatical structures and finding lexical similarities among languages [86], [87], event/information/relation extraction [88], [89] and question answering system [90]. In recent times, we have seen a lot of PoS tagging systems built up for the Urdu language. Works presented by Jawaid and Bojar [91], and Khan et al. [92] show a good survey in this regard.

For the PoS analysis, we get our dataset tagged through the UDPipe¹⁰ library along with the Universal Dependencies¹¹ (UD). The UDPipe presents datasets and UD modules for almost all major languages of the world under the generalized tag set namely, ‘Universal PoS’ (UPoS)¹² [93]. Moreover, the results yielded from UD is in CoNLL-U format[94], [95], that offers various NLP features along PoS, for example lemma of the tokens, dependency relations, and dependency structures etc. Thus, with the PoS tags in

¹⁰<http://ufal.mff.cuni.cz/udpipe>

¹¹<https://universaldependencies.org/>

¹²The UPoS tags presented in Univesal Dependencies are: **ADJ**: Adjective, **ADP**: Adposition, **ADV**: Adverb, **AUX**: Auxiliary, **CCONJ**: Coordinating conjunction, **DET**: Determiner, **INTJ**: Interjection, **NOUN**: Noun, **NUM**: Numeral, **PART**: Particle, **PRON**: Pronoun, **PROPN**: Proper noun, **PUNCT**: Punctuation, **SCONJ**: Subordinating conjunction, **SYM**: Symbol, **VERB**: Verb, and **X**: Other.

CoNLL-U format, we have two kinds of PoS tag sets: UPoS (as mentioned above), and secondly ‘XPoS’, i.e., language-specific PoS tag set i.e., in our case, for Urdu. As far as we could relate the XPoS tag set¹³ employed in UD for Urdu language [96], [97] reflects major contributions from the work of Bharati et al. [98].

In tables V and VI shows the distribution of UPoS and XPoS respectively. Within these tables, the numbers in parenthesis show the percentage of a PoS tag covered in positive, negative, and objective classes. We can see, in both of these tables, that the nouns (NOUN in UPoS, whereas NN*¹⁴ in XPoS) are dominating in the corpus. The second most frequent PoS ad-positions (since, Urdu does not have prepositions; thus, ad-positions, i.e., ADP in table V and PSP in table VI, indicates ad/post-positions instead); followed by the counts of proper nouns, adjectives, verbs, and auxiliaries. We can see the intensifiers (see INTF and INTFC in table VI) dominate the negative class. Similarly, the usage of adverbs and adjectives are dominant in negative class; typically, we assume any corpus is rich if the excellent usage of adverbs and adjectives is made in it; furthermore, enrichment of these PoS tags is seen good for producing better sentiment classifiers [99]. Thus, with the intensifiers, adjectives, and adverbs, and with the word frequencies in negative class (see table I); we can surmise that the people tend to write more enriched contents if they have any negative attitude for something; otherwise, they prefer to write simple contents for positive things.

The figures 10 and 11 show the overall distribution of PoS in UPoS and XPoS tag sets. The y-axis presents the counts of PoS tags in $\log_2$.

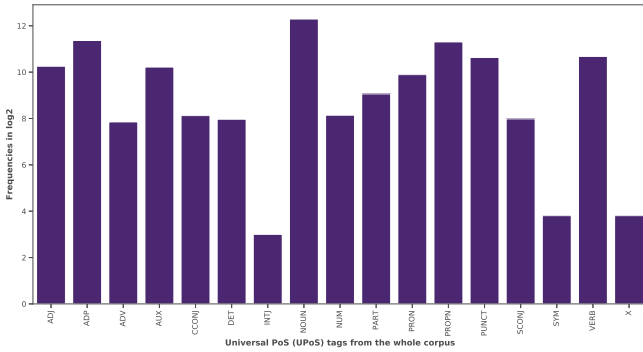


Figure 10: Plot of UPoS tags and their frequencies in corpus.

¹³The Language-Specific PoS (XPoS) for Urdu presented in Universal Dependencies are: **CC**: Coordinating conjunctions, **CCC**: Coordinating conjunction compounds, **DEM**: Demonstrative, **ECH**: Echo, **INJ**: Interjection, **INTF**: Intensifier, **INTFC**: Intensifier compounds, **JJ**: Adjective, **JJC**: Compound Adjective, **NEG**: Negation, **NN**: Common noun, **NN-Ez**: Common noun with Izafat/Ezafat, **NNC**: Common noun compounds, **NNP**: Proper noun, **NNPC**: Proper noun compounds, **NNV**: Verbal noun, **NNZ**: Verbal noun compounds, **NST**: Noun location, **PRP**: Personal pronoun, **PRPC**: Personal pronoun compounds, **PSP**: Postposition, **QC**: Cardinal quantifier, **QCC**: Cardinal quantifier compounds, **QF**: General quantifier, **QO**: Ordinal quantifier, **RB**: Adverb, **RBC**: Adverb compounds, **RDP**: Reduplicative, **RP**: Particle, **SYM**: Symbol, **VAUX**: Verb-auxiliary, **VM**: Verb-finite, **WQ**: Question words, **_**: Unknown.

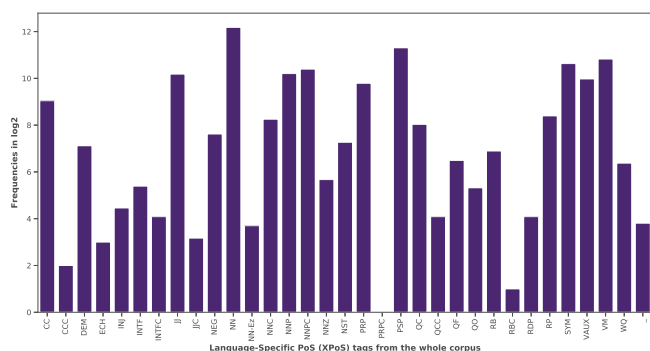
¹⁴NN* collectively shows the sub-types of nouns in XPoS.

Table V: Distribution of Universal Part-of-Speech (UPoS) tags.

PoS Tag	Class			Total
	Positive	Negative	Objective	
ADJ	576 (.4721)	634 (.5197)	10 (.0082)	1220
ADP	1261 (.4782)	1332 (.5051)	44 (.0167)	2637
ADV	102 (.4416)	125 (.5411)	4 (.0173)	231
AUX	531 (.4458)	648 (.5441)	12 (.0101)	1191
CCONJ	141 (.5054)	135 (.4839)	3 (.0108)	279
DET	119 (.4760)	126 (.5040)	5 (.0200)	250
INTJ	4 (.5000)	4 (.5000)	0 (.0000)	8
NOUN	2317 (.4626)	2635 (.5261)	57 (.0114)	5009
NUM	119 (.4220)	155 (.5496)	8 (.0284)	282
PART	232 (.4345)	298 (.5581)	4 (.0075)	534
PRON	455 (.4789)	483 (.5084)	12 (.0126)	950
PROPN	1361 (.5386)	1103 (.4365)	63 (.0249)	2527
PUNCT	719 (.4531)	826 (.5205)	42 (.0265)	1587
SCONJ	121 (.4783)	129 (.5099)	3 (.0119)	253
SYM	8 (.5714)	2 (.1429)	4 (.2857)	14
VERB	769 (.4698)	849 (.5186)	19 (.0116)	1637
X	6 (.4286)	8 (.5714)	0 (.0000)	14

Table VI: Distribution of Language-Specific Part-of-Speech (XPoS) tags.

PoS Tag	Class			Total
	Positive	Negative	Objective	
CC	259 (.4905)	263 (.4981)	6 (.0114)	528
CCC	3 (.7500)	1 (.2500)	0 (.0000)	4
DEM	60 (.4317)	76 (.5468)	3 (.0216)	139
ECH	0 (.0000)	8 (1.0000)	0 (.0000)	8
INJ	10 (.4545)	12 (.5455)	0 (.0000)	22
INTF	20 (.4762)	22 (.5238)	0 (.0000)	42
INTFC	4 (.2353)	13 (.7647)	0 (.0000)	17
JJ	549 (.4729)	605 (.5211)	7 (.0060)	1161
JJC	4 (.4444)	4 (.4444)	1 (.1111)	9
NEG	77 (.3909)	119 (.6041)	1 (.0051)	197
NN	2136 (.4606)	2448 (.5279)	53 (.0114)	4637
NN-Ez	4 (.3077)	9 (.6923)	0 (.0000)	13
NNC	146 (.4787)	155 (.5082)	4 (.0131)	305
NNP	618 (.5237)	534 (.4525)	28 (.0237)	1180
NNPC	743 (.5520)	568 (.4220)	35 (.0260)	1346
NNZ	29 (.5686)	22 (.4314)	0 (.0000)	51
NST	79 (.5130)	72 (.4675)	3 (.0195)	154
PRP	430 (.4859)	446 (.5040)	9 (.0102)	885
PRPC	1 (1.0000)	0 (.0000)	0 (.0000)	1
PSP	1208 (.4769)	1284 (.5069)	41 (.0162)	2533
QC	112 (.4275)	142 (.5420)	8 (.0305)	262
QCC	7 (.4118)	10 (.5882)	0 (.0000)	17
QF	51 (.5667)	38 (.4222)	1 (.0111)	90
QO	18 (.4500)	20 (.5000)	2 (.0500)	40
RB	50 (.4202)	65 (.5462)	4 (.0336)	119
RBC	1 (.5000)	1 (.5000)	0 (.0000)	2
RDP	9 (.5294)	8 (.4706)	0 (.0000)	17
RP	155 (.4599)	179 (.5312)	3 (.0089)	337
SYM	719 (.4531)	826 (.5205)	42 (.0265)	1587
VAUX	452 (.4489)	547 (.5432)	8 (.0079)	1007
VM	848 (.4670)	945 (.5204)	23 (.0127)	1816
WQ	31 (.3735)	48 (.5783)	4 (.0482)	83
_	8 (.5714)	2 (.1429)	4 (.2857)	14



- [34] K. Ahmad, *Affective computing and sentiment analysis: Emotion, metaphor and terminology*. Springer Science & Business Media, 2011, vol. 45.
- [35] E. Cambria, "Affective computing and sentiment analysis," *IEEE Intelligent Systems*, vol. 31, no. 2, pp. 102–107, 2016.
- [36] M. Y. Khan, S. M. Emaduddin, and K. N. Junejo, "Harnessing english sentiment lexicons for polarity detection in urdu tweets: A baseline approach," in *2017 IEEE 11th International Conference on Semantic Computing (ICSC)*. IEEE, 2017, pp. 242–249.
- [37] C. J. Hutto and E. Gilbert, "Vader: A parsimonious rule-based model for sentiment analysis of social media text," in *Eighth international AAAI conference on weblogs and social media*, 2014.
- [38] D. Vilares, M. A. Alonso, and C. Gómez-Rodríguez, "Supervised sentiment analysis in multilingual environments," *Information Processing & Management*, vol. 53, no. 3, pp. 595–607, 2017.
- [39] C. Catal and M. Nangir, "A sentiment classification model based on multiple classifiers," *Applied Soft Computing*, vol. 50, pp. 135–141, 2017.
- [40] W. Muhamamd, M. Mushtaq, K. N. Junejo, and M. Y. Khan, "Sentiment analysis of product reviews in the absence of labelled data using supervised learning approaches," *Malaysian Journal of Computer Science*, 2019.
- [41] X. Hu, J. Tang, H. Gao, and H. Liu, "Unsupervised sentiment analysis with emotional signals," in *Proceedings of the 22nd international conference on World Wide Web*. ACM, 2013, pp. 607–618.
- [42] T. Zagibailov and J. Carroll, "Unsupervised classification of sentiment and objectivity in chinese text," in *Proceedings of the Third International Joint Conference on Natural Language Processing: Volume-1*, 2008.
- [43] S. Baccianella, A. Esuli, and F. Sebastiani, "Sentiwordnet 3.0: an enhanced lexical resource for sentiment analysis and opinion mining," in *Lrec*, vol. 10, no. 2010, 2010, pp. 2200–2204.
- [44] S. Li, Z. Wang, G. Zhou, and S. Y. M. Lee, "Semi-supervised learning for imbalanced sentiment classification," in *Twenty-Second International Joint Conference on Artificial Intelligence*, 2011.
- [45] F. H. Khan, S. Bashir, and U. Qamar, "Tom: Twitter opinion mining framework using hybrid classification scheme," *Decision Support Systems*, vol. 57, pp. 245–257, 2014.
- [46] L. Xie, M. Zhou, and M. Sun, "Hierarchical structure based hybrid approach to sentiment analysis of chinese micro blog and its feature extraction," *Journal of Chinese information processing*, vol. 26, no. 1, pp. 73–83, 2012.
- [47] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.
- [48] W. A. Scott, "Reliability of content analysis: The case of nominal scale coding," *Public opinion quarterly*, pp. 321–325, 1955.
- [49] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and psychological measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [50] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (mae) over the root mean square error (rmse) in assessing average model performance," *Climate research*, vol. 30, no. 1, pp. 79–82, 2005.
- [51] G. K. Zipf, "The psychology of language," in *Encyclopedia of psychology*. Philosophical Library, 1946, pp. 332–341.
- [52] —, "Human behavior and the principle of least effort," 1949.
- [53] D. M. Powers, "Applications and explanations of zipf's law," in *Proceedings of the joint conferences on new methods in language processing and computational natural language learning*. Association for Computational Linguistics, 1998, pp. 151–160.
- [54] W. Li, "Zipf's law everywhere," *Glottometrics*, vol. 5, pp. 14–21, 2002.
- [55] X. Gabaix, "Zipf's law for cities: an explanation," *The Quarterly journal of economics*, vol. 114, no. 3, pp. 739–767, 1999.
- [56] C. Manning, P. Raghavan, and H. Schütze, "Introduction to information retrieval," *Natural Language Engineering*, vol. 16, no. 1, pp. 100–103, 2010.
- [57] W. N. Francis and H. Kucera, "Brown corpus," *Department of Linguistics, Brown University, Providence, Rhode Island*, vol. 1, 1964.
- [58] S. Fagan and R. Gençay, "An introduction to textual econometrics," *Handbook of empirical economics and finance*, pp. 133–154, 2011.
- [59] M. Kirsten, "The linguistics encyclopedia," 2002.
- [60] P. Jaccard, "Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines," *Bull Soc Vaudoise Sci Nat*, vol. 37, pp. 241–272, 1901.
- [61] T. J. Sørensen, *A method of establishing groups of equal amplitude in plant sociology based on similarity of species content and its application to analyses of the vegetation on Danish commons*. I kommission hos E. Munksgaard, 1948.
- [62] L. R. Dice, "Measures of the amount of ecologic association between species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.
- [63] A. Tversky, "Features of similarity," *Psychological review*, vol. 84, no. 4, p. 327, 1977.
- [64] D. J. Rogers and T. T. Tanimoto, "A computer program for classifying plants," *Science*, vol. 132, no. 3434, pp. 1115–1118, 1960.
- [65] A. E. Monge, C. Elkan *et al.*, "The field matching problem: Algorithms and applications," in *KDD*, vol. 2, 1996, pp. 267–270.
- [66] M. Suffian, M. Y. Khan, and S. Wasi, "Developing disease classification system based on keyword extraction and supervised learning," *International Journal of Advanced Computer Science and Applications*, vol. 9, no. 9, pp. 599–605, 2018.
- [67] W. W. R. Ball, *A short account of the history of mathematics*. Courier Corporation, 1960.
- [68] L. Schläfli and H. Wild, *Theorie der vielfachen Kontinuität*. Springer-Verlag, 2013, vol. 38.
- [69] S. T. Roweis and L. K. Saul, "Nonlinear dimensionality reduction by locally linear embedding," *science*, vol. 290, no. 5500, pp. 2323–2326, 2000.
- [70] M. Belkin and P. Niyogi, "Laplacian eigenmaps and spectral techniques for embedding and clustering," in *Advances in neural information processing systems*, 2002, pp. 585–591.
- [71] J. W. Sammon, "A nonlinear mapping for data structure analysis," *IEEE Transactions on computers*, vol. 100, no. 5, pp. 401–409, 1969.
- [72] K. Pearson, "Liii. on lines and planes of closest fit to systems of points in space," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, no. 11, pp. 559–572, 1901.
- [73] H. Hotelling, "Analysis of a complex of statistical variables into principal components," *Journal of educational psychology*, vol. 24, no. 6, p. 417, 1933.
- [74] M. A. Kramer, "Nonlinear principal component analysis using autoassociative neural networks," *AIChE journal*, vol. 37, no. 2, pp. 233–243, 1991.
- [75] Y. Bengio, M. Monperrus, and H. Larochelle, "Nonlocal estimation of manifold structure," *Neural Computation*, vol. 18, no. 10, pp. 2509–2528, 2006.
- [76] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.
- [77] Student, "The probable error of a mean," *Biometrika*, pp. 1–25, 1908.
- [78] S. Kullback and R. A. Leibler, "On information and sufficiency," *The annals of mathematical statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [79] K. W. Church and P. Hanks, "Word association norms, mutual information, and lexicography," *Computational linguistics*, vol. 16, no. 1, pp. 22–29, 1990.
- [80] G. Salton and M. J. McGill, *Introduction to modern information retrieval*. mcgraw-hill, 1983.
- [81] A. Rajaraman and J. D. Ullman, *Mining of massive datasets*. Cambridge University Press, 2011.
- [82] K. N. Junejo and A. Karim, "A robust discriminative term weighting based linear discriminant method for text classification," in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 323–332.
- [83] K. N. Junejo, A. Karim, M. T. Hassan, and M. Jeon, "Terms-based discriminative information space for robust text classification," *Information Sciences*, vol. 372, pp. 518–538, 2016.
- [84] S. M. Mohammad, S. Kiritchenko, and X. Zhu, "Nrc-canada: Building the state-of-the-art in sentiment analysis of tweets," *arXiv preprint arXiv:1308.6242*, 2013.
- [85] J. Blogowska and V. Saroglou, "For better or worse: Fundamentalists' attitudes toward outgroups as a function of exposure to authoritative religious texts," *International Journal for the Psychology of Religion*, vol. 23, no. 2, pp. 103–125, 2013.
- [86] M. S. Nizami, M. Y. Khan, and T. Ahmed, "Towards a generic approach for pos-tagwise lexical similarity of languages," in *Proceedings of the 2nd International Conference on Intelligent Technologies and Applications (INTAP)*. Springer, 2019.
- [87] T. Ahmed, M. S. Nizami, and M. Y. Khan, "Discovering lexical similarity through articulatory feature-based phonetic edit distance." Submitted, 2020.

- [88] H. Schütze, C. D. Manning, and P. Raghavan, "Introduction to information retrieval," in *Proceedings of the international communication of association for computing machinery conference*, vol. 4, 2008.
- [89] K. Iqbal, M. Y. Khan, S. Wasi, S. Mahboob, and T. Ahmed, "On extraction of event information from social text streams: An unpretentious nlp solution," *IJCSNS*, vol. 19, no. 9, p. 1, 2019.
- [90] D. Jurafsky, *Speech & language processing*. Pearson Education India, 2000.
- [91] B. Jawaid and O. Bojar, "Tagger voting for urdu," in *Proceedings of the 3rd Workshop on South and Southeast Asian Natural Language Processing*, 2012, pp. 135–144.
- [92] W. Khan, A. Daud, K. Khan, J. A. Nasir, M. Basher, N. Aljohani, and F. S. Alotaibi, "Part of speech tagging in urdu: Comparison of machine and deep learning approaches," *IEEE Access*, vol. 7, pp. 38 918–38 936, 2019.
- [93] J. Nivre, M.-C. De Marneffe, F. Ginter, Y. Goldberg, J. Hajic, C. D. Manning, R. McDonald, S. Petrov, S. Pyysalo, N. Silveira *et al.*, "Universal dependencies v1: A multilingual treebank collection," in *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, 2016, pp. 1659–1666.
- [94] J. Nivre, "Towards a universal grammar for natural language processing," in *CICLing*, 2015.
- [95] Ç. Çöltekin, B. Campbell, E. Hinrichs, and H. Telljohann, "Converting the tüba-d/z treebank of german to universal dependencies," in *Proceedings of the NoDaLiDa 2017 Workshop on Universal Dependencies (UDW 2017)*, 2017, pp. 27–37.
- [96] R. A. Bhat, R. Bhatt, A. Farudi, P. Klassen, B. Narasimhan, M. Palmer, O. Rambow, D. M. Sharma, A. Vaidya, S. R. Vishnu *et al.*, "The hindi/urdu treebank project," in *Handbook of Linguistic Annotation*. Springer Press.
- [97] M. Palmer, R. Bhatt, B. Narasimhan, O. Rambow, D. M. Sharma, and F. Xia, "Hindi syntax: Annotating dependency, lexical predicate-argument structure, and phrase structure," in *The 7th International Conference on Natural Language Processing*, 2009, pp. 14–17.
- [98] A. Bharati, R. Sangal, D. M. Sharma, and L. Bai, "Anncorra: Annotating corpora guidelines for pos and chunk annotation for indian languages," *LTRC-TR31*, pp. 1–38, 2006.
- [99] F. Benamara, C. Cesarano, A. Picariello, D. R. Recupero, and V. S. Subrahmanian, "Sentiment analysis: Adjectives and adverbs are better than adjectives alone," in *ICWSM*. Citeseer, 2007, pp. 1–7.
- [100] M. Y. Khan and T. Ahmed, "Pseudo transfer learning by exploiting monolingual corpus: An experiment on roman urdu transliteration," in *Proceedings of the 2nd International Conference on Intelligent Technologies and Applications (INTAP)*. Springer, 2019.